COMP

411/511

Project6: Jam with Continuations

Points: 100

Provided Code

The **src** directory in your initial repository includes the two JUnit test files **Assign6Test.java** and **Assign4Test.java**. The former contains some simple unit tests for Project 6 that identify the API that your

solution must support. This API is also described below, but the executable nature of **Assign6Test.java** makes conformance easy to check. The file **Assign4Test.java** contains some older Jam tests that are compatible with Project 6. The **src** directory also includes the skeleton of the existing class solution to Project 6. You are free to use this code or ignore it in favor of writing you own code. The skeleton code is written so that it compiles and passes all of the tests in **Assign6Test.java** that are not commented out. These commented out tests are perhaps more important than the currently executable test methods because they invoke methods that are stubbed out in the skeleton solution. If you use the skeleton solution as a starting point, you will need to fill in all of the stubbed methods. If you use your solution to Step 1 of Project 5 as the starting point, you need to pass all of the tests in **Assign6Test.java** including those that are commented out. You are free to use whatever subset of the skeleton solution that you like.

If you understand the programming language concepts underlying this assignment, it is longer than previous assignments but it is not conceptually difficult. To fill in all of the stubbed methods in the provided skeleton solution, you need to write on the order of 700 lines of

Java code. The most delicate part is correctly implementing the CPS transformation rules. Small clerical mistakes can be very time-consuming to debug. We strongly recommend the strategy of careful test-driven program development in writing your solution. The CPS transformation rules are precisely stated below.

Overview

Task Your assignment is:

1. to transform an untyped, call-by-value, eager dialect of Jam to *continuation passing style* (*CPS*);

2. to support the conversion of conventional symbolic **AST**s (henceforth called **SymAST**s) to use *static distance coordinates* which requires a new syntax tree representation (sharing many **AST** classes with **SymAST**) called **SDAST**;

3. to write an interpreter for the **SDAST** representation analogous to the existing interpreter for the **SymAST** representation in the support code; and

4. to extend your implementation of the CPS transformation from step 1. to support the addition of the **letcc** construct to Jam. The **letcc** construct is formally defined below.

The skeleton solution identifies a collection of methods that if implemented correctly (perhaps depending on additional private methods) constitute a solution to this project. This program architecture of the skeleton is not as elegant as the class solution to Project 3, which has recently been refactored to eliminate some unnecessary present in the skeleton solution for this project. Ambitious students may want to improve on the architecture in the provided code. (Look at the architecture of the class solution to Project 3 for some potential refactorings.)

The Jam dialect for Project 6 is restricted to call-by-value/eager semantics. The only addition to the source language from Projects 4 is the presence of two different forms of "let" binding: unrestricted ordinary **let** exactly as in Assignment 2 and **letrec** (recursive "let") as in Assignment 3, except that this form of “let” is denoted by keyword **letrec** and the right hand sides of the **letrec** bindings are restricted to **map** constructions. These small changes to the call-by-value/eager version of Jam in Project 4 simplify the

rules for conversion to **CPS**. The parser in the skeleton solution supports both of these "let" constructs including the **map** restriction on **letrec**. In the last phase of the project, you will extend the supported Jam dialect to include the **letcc** construct, which implemented simply by extended the **CPS** converter. No changes to the interpreters are involved because the **letcc AST** node is eliminated in the **CPS** conversion process.

The support code includes a call-by-value/eager interpreter for Jam **SymAST**s as defined below for this assignment. This Jam dialect is essentially the same language as the call-by-value/eager formulation of Jam in Assignment 4. The support code includes a revised parser tailored to this language plus a framework for doing the assignment. Given this support code, we can restate the four phases of the assignment as follows: (1) completing the implementation of the method **convertToCPS** in the **Interpreter** class that transforms the embedded Jam program to **CPS**; (2) completing the implementation of the method **convertToSD** in the **Interpreter** class to transform the embedded program represented as an **SymAST** to the corresponding **SDAST**; (3) completing the implementation of the method **SDEval**

in the **Interpreter** class to interpret the embedded Jam program represented as an **SDAST** analogous to the **eval** represented as an **SymAST** in the support code; and (4) supporting the addition of the **letcc** construct to Jam by using the **CPS** transformation to eliminate **letcc** from Jam programs.

The **Parser** provided in our support code correctly parses **letcc** but the **SymAST** interpreter aborts with an error if you try to directly execute it. The CPS transformation augmented by rules given below (**forLetcc** converts Jam programs with **letcc** to equivalent, more complex Jam programs without **letcc**, so our **Interpreter** class can support Jam programs using **letcc** by first converting the program to **CPS** and interpreting the transformed program. To make this assignment more manageable, the course staff has provided a skeleton solution code (excerpted from the class solution) consisting of a solution to this Assignment except for the principal methods in the visitors that perform **CPS** conversion, **SD** conversion, and **SD** interpretation. All of these methods have degenerate bodies (typically returning **null**) that are marked with a comment including the word **STUB** which you can search for using your IDE or text editor. The support code compiles as is under Java 8

and the unit tests, except for those that are commented out, all pass. Your complete solution should pass all of the tests, including those that are currently commented out.

This is a more challenging assignment than the previous assignments and hence is worth 150 points instead of 100 points.

Background The Jam language for this assignment is identical to the call-by-value, eager language used in Project 4 with two tweaks. First, in Project 6 Jam, there are two let constructs: (i) **letrec**, which is recursive let with right hand sides limited to **map** constructions, and (ii) **let**, which is ordinary non-recursive let abbreviating the application of a map to the right hand sides. These modifications to the Jam language simplify the definition of the **CPS** transformation. This "tweak" revises the definition of the language syntax as follows:

Expressions

```
Exp      ::= ...
                 | let Def+ in Exp
                 | letrec MapDef+ in Exp
                 | Map
```

| ...

Map ::= map IdList to Exp
Definitions

Def ::= Id := Exp ;
MapDef ::= Id := Map ;

Second, primitive functions are *not* values in Project 6 Jam. They have exactly the same semantics as unary and binary operators except that they are written using conventional function application notation. Hence, to pass **first** as a function, you must pass the **map**

map x to first(x)
instead. In the project, the only meaning for Jam provided in the **Interpreter** class is call-by-eager/ eager semantics. The **valueValue()** method from Project 4. is renamed **eval()** and the other evaluation options from Project 4 are discarded. In Project 6, you must also support an interpreter with the same semantics as **eval()** that uses a slightly different form of **AST** called an **SDAST** that uses static distance coordinates instead of conventional symbolic variables. The actual differences between the types **SDAST** and **SymAST** are small. Look at the

defintiions of the **SymAST** classes **Variable**, **Map**, **Let**, and **LetRec** versus the **SDAST** classes **SDPair**, **Smap, SLet**, and **SletRec**. In hindsight, we could have used a single albeit more complex abstract syntax essentially consisting of the **AST** classes from Project 4 augmented by some extra fields and methods to support the static distance representation.

Support Code Preliminaries The skeleton solution already outlines a transformer that prevents shadowing variable names by converting the name of each Jam variable from ***x*** to ***x*:*d*** where ***d*** is the lexical depth of the declaration of ***x*** in the program. This renaming is performed by the **CheckVisitor** operation that checks for syntax errors that are not part of the context free grammar specification for Jam. If a variable ***x*** is free in the input program (not allowed in legal programs) then it has lexical depth 0. Hence, in a legal Jam program, no variable ever has depth 0. If a variable is introduced (the binding occurrence) inside one level of lexical nesting, it has lexical depth 1. For example, the program

(map *x* to *x*)(7)
becomes

(map *x*:1 to *x*:1)(7)

after renaming. Similarly,

let *id* := map *y* to *y*; in *id*(1)
becomes

let *id*:1 := map *y*:1 to *y*:1; in *id*:1(1)
after renaming. The right-hand-sides of **let** bindings *have the same nesting level* as the surrounding context. Recall the expansion of **let** in terms of **map**. But the right-hand-sides of **letrec** bindings *have a nesting level one greater* than the surrounding context.

Renamed variables cannot be confused with existing variable names because the character `:` is not a legal character in variable names read by the parser. Within a given scope, all variables in the lexical environment must have distinct names because all variables introduced in a particular **let** or **map** must be distinct and the names at every nesting level are disjoint because of the added `:` suffixes.

The skeleton solution includes a method

public AST unshadow();
in the **Interpreter** class that returns the unshadowed **SymAST** for the input program after applying the **CheckVisitor** that checks for non-context-free syntax

errors while performing the unshadowing transformation. Note that the unshadowing transformation is applied immediately after the input program (a file of characters) is parsed into a **SymAST**, regardless of whether or not the CPS transform is later applied. Syntax checking and unshadowing are bundled together.

The unshadowing transformation permits **let** constructs to be re-interpreted as **let*** without changing the meaning of programs. The **let*** construct is a variant of **let** supported in Scheme/Racket and other languages, which has been dicussed in class lecture. In addiiton, a formal definition of **let*** (as syntactic sugar for a chain of nested **let** constructions) is given in the the sample mid-term exam posted on the main course webpage. The correctness of our rules for **CPS** transformation hinges on this identity. But no explicit use of **let*** semantics appears in this project. (The unshadowing eliminates the semantic distinction between **let** and **let***.)

Note: In order to support the full **CPS** transformation without changing the semantics of the input program, we need to ensure that "short-circuit" evaluation is performed in **CPS**ed programs for the boolean **&** and **|**

operators. The **CPS** rules assume that all primitive functions evaluate all of their arguments which conflicts with short-circuit evaluation. The provided support code already works around this problem by supporting the primitive function **asBool** to eliminate the **&** and **|** operators as part of syntax checking (as implemented by the **checkProg()** method in **Interpreter** instances). This **asBool** function has a very simple definition. For **asBool(*x*)**:

- If ***x*** is **true** then **asBool(*x*)** is **true**.
- If ***x*** is **false** then **asBool(*x*)** is **false**.
- Otherwise, **asBool(*x*)** generates in an **EvalException.**

The syntax checker in your solution must perform the following transformations on the syntax of input programs

***M* & *N* ® if *M* then asBool(*N*) else false**
***M* | *N* ® if *M* then true else asBool(*N*)**

so that your **CPS** converter does not have handle any primitive functions that do not evaluate all of their arguments. Without these transformations, the **CPS** conversion process (embodied in the rules given below) would treat **&** and **|** just like other binary

operators and always force the evaluation of both arguments (as in call-by-value evaluation of program-defined functions). Note how this transformation prevents **&** or **|** from returning non-boolean values, consistent with their definitions in prior assignments.

Phase 1 Complete the implementation of the method **convertToCPS()** in class **Interpreter** that transforms the embedded unshadowed program to equivalent **CPS** form given a *continuation* ***k*** represented by a Jam **map**. At the top-level, this continuation is the identity function, but the definition of this transformation is recursive, so the continuation may be more complex in recursive calls. In Java, we implement **convertToCPS** with a visitor so the code you actually have to write is the set of key **forXXX** methods in the corresponding visitor class (called **ConvertToCPS** in the support code. Specifically, given a Jam input program ***M'*** (with possible shadowing), your **convertToCPS** operation should return ***Cp*s[map x to x, *M*],** where ***M*** is the unshadowed program (as defined in the support code) corresponding to *M*', ***Cp*s[*k*, *M*]** is the binary function that takes a Jam expression ***k*** denoting a Jam function and a Jam expression ***M*** (both with no free variables and no variable shadowing) and returns the

Jam expression determined by the transformation rules for ***Cps*** given below. These rules are a loosely based on the exposition in Friedman, *et al.*, Chapter 8. In the process, you will need to write the method **checkProg()** based on implementing a **CheckVisitor** class similar to the one in Project 4 that (*i*) checks that programs are syntactically well-formed as in Project 4; (*ii*) eliminates the operators **&** and **|** using the transformations shown above; and (*iii*) unshadows variables as shown above. The code for the version of **CheckVisitor** in this project is only slightly more complex that the code for the same visitor in Project 4; the major addition is unshadowing variable names which requires very little code.

*These rules for **CPS** conversion presume that variables have been renamed to prevent shadowing* ("holes in scope" [nested variables with the same name]). The rules will not work correctly on programs that shadow variable names without the unshadowing transformation performed by the **checkProg** method.

For the purposes of this assignment, we consider operator applications (both unary and binary) as syntactic sugar for applications of corresponding primitive operations. Hence operator applications are

treated just like primitive applications.

If your **CPS** tranformer encounters a **letcc** construct embedded in an input program, it should throw an exception reporting the error and abort execution. We will replace this code by a simple translation of this construct in Phase III. The **letcc** construct is supported by syntactic transformation rather than direct interpretation.

In the support code the **CPS** processor is implemented as the method

public AST convertToCPS()
in the **Interpreter** class. If you write your own solution independent of the skeleton solution, you must implement this method in this class, which is part of the API assumed by our grading tests. You must also support the method (nominally included in the skeleton solution)

public JamVal cpsEval();
in the **Interpreter** that converts the embedded program to **CPS** and then interprets the transformed program using **eval**. The **ConvertToCPS** visitor class in the skeleton solution is stubbed out.

You can test that your implementation of the **CPS**

transformation preserves the meaning of programs in Java by comparing the results produced by **eval()** and **cpsEval()**.

To state the **CPS** transformation rules, we need to introduce a few technical definitions. Study them until you thoroughly understand them.

A Jam application ***E*(*E1*, ...,*En*)** is *primitive* if ***E*** is a primitive Jam function. Recall that we are interpreting operator applications as syntactic sugar for applications of corresponding primitive operations. So an application is primitive *if and only if* the rator of the application is either a primitive function or an operator. For example, the applications **first(append(x,y))** and **square(x) * y** are both primitive applications (but not expressions!) while the applications **square(4)** and **append(x,y)** are not.

A Jam expression ***E*** is *simple if and only if* all applications except those nested inside **map** constructions are primitive, *i.e.*, have a primitive function or operator as the rator. For example,

let x := first(a) * b * first(c);
Y := map f to let g := map z to f(z(z)); in g(g);
in cons(x, cons(Y, null))

and

x+(y*z)
are both simple. In contrast,

f(1)
and

let Y := map f to let g := map z to f(z(z)); in map x to (g(g))(x);
in Y(map fact to map n to if n=0 then 1 else n*fact(n-1))
are not simple because **f** is not primitive and **Y** is not primitive.

The following rules define two syntactic transformers (functions) on Jam program text: the binary transformer ***Cps*** **: Jam * Jam -> Jam** and the unary transformer ***Rsh*** **: Simp -> Simp**, where **Jam** is the set of all Jam expressions and **Simp** is the set of simple Jam expressions (***Rsh*** stands for "reshape"). The binary transformer ***Cps*[*k*,M]** takes a Jam expression ***k*** denoting a unary function, and an unshadowed Jam expression **M** as input and produces a tail-calling (every call on a program-defined function appears in tail position) Jam expression with the same meaning as ***k*(M)**.

The unary transformer ***Rsh*** is a "help" function for ***Cps*** that takes an unshadowed simple expression as input and adds a continuation parameter to the map expressions and function constants embedded in simple expressions. ***Rsh*** also adjusts applications of the **arity** primitive function to ignore the added continuation argument.

In the following rules, ***S***, ***S1***, ***S2***, ... are meta-variables (pattern matching variables) denoting simple Jam expressions; ***A***, ***B***, ***C***, ***E***, ***E1***, ***E2***, ..., ***T*** are meta-variables denoting arbitrary Jam expressions; ***x1***, ***x2***, ... are metavariables denoting ordinary Jam identifiers; ***v***, ***v1***, ***v2***, ... denote fresh Jam identifiers that do not appear in any other program text; and ***k*** is a metavariable denoting a Jam expression that evaluates to a unary function. The variable names ***x***, ***y***, and ***k*** embedded in program text denote themselves.

Definition of *Cps* The following clauses define the textual transformation ***Cps*[*k*, *M*]**:

1. If ***M*** is a simple Jam expression ***S***:
   ***Cps*[*k*, *S*] ® *k*(*Rsh*[*S*])**
2. If ***M*** is an application (**map *x1*, ..., *xn* to *B***) (***E1*, ...,*En***), ***n* > 0**:

***Cps*[*k*, (map *x1*, ..., *xn* to *B*) (*E1*, ...,*En*)] ® *Cps*[*k*, let *x1* := *E1*; ...; *xn* := *En*; in *B*]**

3. If ***M*** is an application **(map to *B*)**(),:
   ***Cps*[*k*, (map to *B*)()] ® *Cps*[*k*, *B*]**
4. If ***M*** is an application ***S*(*S1*, ..., *Sn*), *n* >= 0**:
   ***Cps*[*k*, *S*(*S1*, ..., *Sn*)] ® *Rsh*[*S*] (*Rsh*[*S1*], ...,*Rsh*[*Sn*], *k*)**
5. If ***M*** is an application ***S*(*E1*, ...,*En*), *n* > 0**:
   ***Cps*[*k*, S(*E1*, ...,*En*)] ® *Cps*[*k*, let *v1* := *E1*; ... *vn* := *En*; in *S*(*v1*, ..., *vn*)]**
6. If ***M*** is an application ***B*(*E1*, ...,*En*), *n* >= 0** where ***B*** is not simple:
   ***Cps*[*k*, *B*(*E1*, ..., *En*)] ® *Cps*[*k*, let *v* :=*B*; *v1* := *E1*; ... *vn* := *En*; in *v*(*v1*, ..., *vn*)]**
7. If ***M*** is a conditional construction **if S then *A* else *C***:
   ***Cps*[*k*, if *S* then *A* else *C*] ® if *Rsh*[*S*] then *Cps*[*k*, *A*] else *Cps*[*k*, *C*]**
8. If **M** is a conditional construction **if *T* then *A***

else *C*:

***Cps*[*k*, if *T* then *A* else *C*] ® *Cps*[*k*, let *v* := *T* in if *v* then *A* else *C*]**

9. If ***M*** is a block {***E1***; ***E2***; ...; ***En***}, ***n* > 0**:

   ***Cps*[*k*, {*E1*; *E2*; ...; *En*}] ® *Cps*[*k*, let *v1* := *E1*; ...; *vn* := *En*; in *vn*]**

10. If ***M*** is **let *x1* := *S1*; in *B***:

    ***Cps*[*k*, let *x1* := *S1*; in *B*] ® let *x1* := *Rsh*[*S1*]; in *Cps*[*k*, *B*]**

11. If ***M*** is **let *x1* := *S1*; *x2* := *E2*; ... *xn* := *En*; in *B*, *n* > 1**:

    ***Cps*[*k*, let *x1* := *S1*; *x2* := *E2*; ... *xn* := *En*; in *B*] ® let *x1* := *Rsh*[*S1*]; in *Cps*[*k*, let *x2* := *E2*; ...; *xn* := *En*; in *B*]**

12. If ***M*** is **let *x1* := *E1*; in *B***:

    ***Cps*[*k*, let *x1* := *E1*; in *B*] ® *Cps*[map *x1* to *Cps*[*k*, *B*], *E1*]**

13. If ***M*** is **let *x1* := *E1*; *x2* := *E2*; ... *xn* := *En*; in *B*, *n* > 1**:

    ***Cps*[*k*, let *x1* := *E1*; ... *xn* := *En*; in *B*] ® *Cps*[map *x1* to *Cps*[*k*, let *x2* = *E2*; ... *xn* := *En*; in *B*], *E1*]**

14. If ***M*** is **letrec *p1* := map ... to *E1*; ...; *pn* := map ... to *En*; in *B***:

***Cps*[*k*, letrec *p1* := map ... to *E1*; ...; *pn* := map ... to *En*; in *B*] ®**

letrec *p1* := *Rsh*[map ... to *E1*]; ...; *pn* := *Rsh*[map ... to *En*]; in *Cps*[*k*,*B*]

Definition of *Rsh* The helper transformer ***Rsh*[*S*]** is defined by the following rules:

1. If *S* is a ground constant ***c*** (value that is not a map): ***Rsh*[*c*] ® *c***
2. If ***S*** is a variable ***x***: ***Rsh*[*x*] ® *x***
3. If ***S*** is a primitive application **arity**(***S1***): ***Rsh*[arity(*S1*)] ® arity(*Rsh*[*S1*]) - 1**
4. If ***S*** is a primitive application ***f*(*S1*, ..., *Sn*), *n* >= 0** where ***f*** is not **arity**: ***Rsh*[*f*(*S1*, ..., *Sn*)] ® *f*(*Rsh*[*S1*], ..., *Rsh*[*Sn*])**
5. If ***S*** is **map *x1*, ..., *xn* to E, *n* >= 0**: ***Rsh*[map *x1*, ..., *xn* to *E*] ® map *x1*, ..., *xn*, *v* to *Cps*[*v*, *E*]**
6. If ***S*** is the primitive function **arity**: ***Rsh*[arity] => map x,k to k(arity(x) - 1)**
7. If ***S*** is a unary primitive function ***f*** other than **arity**: ***Rsh*[*f*] ® map x,k to k(*f*(x))**

8. If ***S*** is a binary primitive function ***g***: ***Rsh*[*g*]** ® **map x,y,k to k(*g*(x,y))**

9. If ***S*** is a conditional construct **if *S1* then *S2* else *S3***:
   ***Rsh*[if *S1* then *S2* else *S3*]** ® **if *Rsh*[*S1*] then *Rsh*[*S2*] else *Rsh*[*S3*]**

10. If **S** is **let *x1* := *S1*; ...; *xn* := *Sn*; in *S*, *n* > 0**:
    ***Rsh*[let x1 := *S1*; ...; *xn* := *Sn*; in *S*]** ® **let *x1* := *Rsh*[*S1*]; ...; *xn* := *Rsh*[*Sn*]; in *Rsh*[*S*]**

11. If **S** is **letrec *p1* := map ... to *E1*; ...; *pn* := map ... to *En*; in *S*, *n* > 0**:
    ***Rsh*[letrec *p1* := map ... to *E1*; ...; *pn* := map ... to *En*; in *S*]** ®
    letrec *p1* := *Rsh*[map ... to *E1*]; ...; *pn* := *Rsh*[map ... to *En*]; in *Rsh*[*S*]

12. bIf ***S*** is a block **{*S1*; ...; *Sn*}**, ***n* > 0**:
    ***Rsh*[{*S1*; ...; *Sn*}]** ® **{*Rsh*[*S1*]; ...; *Rsh*[*Sn*]}**

For the purposes of testing your programs we require the following standardization. The top-level continuation must have exactly the syntactic form

map *x* to *x*

using the variable name ***x***. In some transformations, you must generate a new variable name. For this purpose, use variable names of the form ***:i*** where ***i*** is an integer. These names cannot be confused with the names of variables that already exist in the program. The sequence of variable names generated by your CPS transformer must be **:0**, **:1**, **:2**, ... so that your CPS transformer has exactly the same behavior as our solution. Note that you must transform a program by making the *leftmost possible transformation* given that match variables ***S*** and ***E*** can only match raw program text (any embedded calls on ***Cps*** and ***Rsh*** must have already been reduced).

Some of the code required to implement the **CPS** transformation is already included in the skeleton solution. All that you need to do is to write the "stubbed" methods (most of the code implementing the **CPS** transformation) in the visitor that converts a **SymAST** to **CPS** form.

You are welcome to modify your solution to Assignment 4 instead of using the support code framework, but make sure that you can pass all of the tests in the test files included in the support code. These tests use the same API that our grading tests

require.

Phase 2 The second part of the assignment is straightforward. You have to complete the implementation of the method **convertToSD** in the **Interpreter** class. If you use the skeleton solution, you only have to implement the stubbed methods in the **SConvert** visitor class encoding a recursive function of the **SymAST** to be converted, the lexical depth of this **SymAST** within the program, and a symbol table mapping free symbolic variables to the definition depth (in the program) and their offset. The only funky aspect of this phase (and the next) is the fact that class for representing static distance variables in the support code is named **Pair** because a static distance coordinate is simply a pair of non-negative integers: the number of lexical levels outside the current level and the offset within a record (assuming every **JamVal** has an offset of 1). All counting starts with 0, so the first variable in the local lexical scope is **[0:0]**. The **SConvert** visitor is an instance of an inner class where the enclosing instance contains a symbol table, a mutable Java **HashMap**. In a purely functional solution, the symbol table would be an immutable value passed to the **SConvert** visitor (or function), but efficiently treating hash maps as values

looked more painful than carefully performing mutation on a single Java **HashMap**. Note that variable references are *relative*, a reference specifies a variable by a **Pair** consisting of the number levels outside the current level identifying the variable's definition (binding occurrence) and the offset of the variable within that level (which can introduce any finite number of variables).

Phase 3 The third part of the assignment is perhaps even easier than Phase 2. You only have to complete the implementation of the method **SDEval** in the **Interpreter** class. If you use the support code, you only have to implement five **forXXX** methods in the **SDEvaluator** visitor class. Many **forXXX** methods in **SDEvaluator** are shared with the visitor implementing the **eval** method for **SymASTs** and hence provided with the support code. Each **forXXX** method simply performs what the corresponding method does in the **SymEvaluator** visitor in the support code implementing the evaluation of **SymAST** programs; only the representation of variables and environments is different.

Phase 4 As the fourth part of the assignment, extend your **CPS** transformer from Phase 2 to handle the

letcc construct. This language extension is often called *supporting first-class continuations*. This phase only involves a few lines of code. Since the support code already includes **forLetcc** method in the **SymASTVisitor** interface, you may have already written this code in Phase 1 instead of leaving it stubbed out.

As we have already observed, the provided parser handles the **letcc** construct but the provided interpreter does not and aborts with an error if it encounters such a construct. In this phase, we officially extend the source language to include the new construct.

Exp ::= ... | letcc *x* in *M*

The new construct **letcc *x* in *M*** binds the identifier ***x*** to the current continuation, and evaluates ***M*** in the extended environment. A continuation is a closure of one argument, reshaped to take an auxiliary continuation argument (like all other closures after conversion to **CPS)** which unlike other closures, it discards. Invoking the continuation ***k*** on a return value replaces the current continuation by ***k*** and applies ***k*** to the specified value and the current (soon to be discarded) continuation. Since continuations are ordinary values, they can be stored in data structures,

passed as arguments, and returned as values. There is a large Scheme literature (largely written in the 1980s) on various programming operations (such as co-routines) that can be performed using continuations.

In our prior implementations of Jam, the **letcc** construct is only supported in the interpreters augmented by **CPS** conversion of the code. The conventional interpreters abort execution with an error if they encounter a use of **letcc**. To perform **CPS** conversion on program containing **letcc**, we extend our rules for **CPS** conversion as follows.

First, a Jam expression **`E`** is *simple if and only if* all occurrences of the **letcc** construct and non-primitive applications appear nested within **`map`** constructions.

Second, we add the following clause to the definition of the ***`Cps`*** syntax transformer:

- If ***`M`*** is **letcc *`x1`* in *`B`*:**

  ***`Cps`*[*`k`*, letcc *`x1`* in *`B`*] ® let *`x1`* := map *`v`*, *`v1`* to *`k`*(*`v`*) in Cps[*`k`*,*`B`*]**

Testing You are expected to write unit tests for all of your non-trivial methods or functions *and submit your test code as part of your program*. For more

information, refer back to Assignment 1.

Testing and Submitting Your Program

The **src** folder in the initial GitHub repository for the assignment contains a skeleton solution including a modest JUnit test file called **Assign6Test.java** covering the API that your solution must support. You will need to write many more tests. You need to test each **CPS** conversion rule and a diverse set of small example Jam programs for your **SDAST** converter and interpreter. One good test of **SD** conversion and interpretation is to evaluate a diverse set of Jam programs taken from your unit test for earlier Jam assignments (except for the fact that only call-by-value/eager evaluation is supported in Assignments 6 and 7) and compare the output from ordinary high-level interpretation, **SD** interpretation of the **SD** version, and ordinary interpretation of the **CPS** conversion of the original program. You could even perform **SD** evaluation of the **SD** conversion of the **CPS** conversion of a program to confirm that the meaning of the original program (under **SymAST** interpretation) is preserved. Of course, this merely confirms that the **CPS** conversion process preserves semantics. It does not confirm that it precisely

implements our rules. (In principle, we could relax the contract for **CPS**ed code so that it preserves semantics and always puts calls on program defined functions is tail position, but these properties are difficult to test--particularly semantics preservation, so our grading suite simply tests that you have correctly implemented the specified syntactic transformation rules.

As always, each class and method in your program should be preceded by a short j**avadoc** comment stating precisely what it does.

Create a **README** file in the your repository that

- outlines the organization of your program,
- specifies any compatible additions that you made the provided API (not generally necessary), and
- specifies what testing process you used to confirm the correctness of your program.

Please make sure to remove or disable all debugging output generated by program. Excessive use of print statements considerably slows down your interpreter and thus the testing of your project. If your interpreter generates a timeout error because debug output drastically slows it down, you may not get credit for the test cases that took too long.